

Exo Runtime AI Agent Guide

Exo Runtime AI Agent Guide

This file is the entry point for any AI agent working on this project. Read this first, then follow the routes below to the detailed documentation.

Project Overview

Exo Runtime

is a local, offline AI coding agent built in Rust. It runs a 1.7B parameter language model (SmolLM2-1.7B Q8_0) via `llama.cpp` on severely constrained hardware: an Intel i5 6th gen, 8GB RAM, mechanical HDD, Linux Mint XFCE.

The Rust binary (~3.2 MB) gives the model 7 safe, bounded filesystem tools. No shell execution. No network access beyond localhost. No GPU required.

Repository Files (Read in This Order)

File	What It Contains	Should You Read It?
`guide.md`	This file. Entry point. Routes to other docs.	[Yes] Always
`README.md`	High-level summary for humans. Tool table, resource budget, quick start.	[Yes] If unfamiliar
`exo-runtime-blueprint.md`	Architecture philosophy, design decisions, failure modes, what to create. No code.	[Yes] Before making changes
`approach.md`	The main document. Full Rust source code, system tuning, llama.cpp config, AppArmor/nftables security, production runbook, Mermaid diagrams, testing strategy, quick-start checklist.	[Yes] Always this is the source of truth
`response.md`	Early architectural critique. Historical context only.	[No] Skip unless investigating old decisions

How an AI Should Use This Project

Step 1: Read the Source of Truth

Read

`approach.md`

in full. It contains the complete implementation:

- **Section 1:**

The Rust code (Cargo.toml + main.rs with ContextWindow, JSON extractor, 7 tools, path validator, HTTP client, signal handler, context persistence)

- **Section 2:**

Unit test design (21 `validate_workspace_path` tests, 16 `extract_json` tests, 15+ `parse_tool_call` tests, integration scenario)

- **Section 3:**

Linux Mint system tuning (sysctl, I/O scheduler, CPU governor, fstab, swap/zram)

- **Section 4:**

llama.cpp server configuration (systemd unit, flags, health check)

- **Section 5:**

Exo Runtime configuration (env vars, build profile)

- **Section 6:**

Security hardening (AppArmor profiles for both binaries, nftables, bind mount, disk quota, sudoers)

- **Section 7:**

Production runbook (start/stop scripts, monitoring dashboard, logrotate, log viewing)

Step 2: Understand the Architecture

Read

`exo-runtime-blueprint.md`

for the architectural philosophy why Rust over Python, why typed tools over bash, why Q8_0 over other quantizations, why

`minreq`

over
`reqwest`

.

Step 3: Make Changes

All changes should be made to

approach.md

Section 1 (Rust code) or the relevant section (system tuning, security, etc.). Keep the document as the single source of truth.

Project State

- **Code:**
Complete. All known bugs fixed through multiple critique cycles. Ready to compile.
- **Deployment:**
Not yet deployed. Awaits migration from Windows to Linux Mint.
- **Testing:**
Unit test cases documented in Section 2. No test harness manual verification on target hardware.
- **Security:**
AppArmor + nftables + bind mount + quota. Four layers of defense in depth.

Applicable Skills

When working on this project, the following skills are relevant:

Skill	When to Use
diagnosing-bugs	Debugging Rust compilation errors, runtime panics, or logic bugs in the approach.md code
investigate	Systematic root cause analysis for hard bugs (HTTP retry issues, path validation edge cases, UTF-8 boundary panics)
karpathy-guidelines	Writing or reviewing Rust code to avoid overcomplication, make surgical changes, and define verifiable success criteria
codebase-design	Evaluating module boundaries the ContextWindow, JSON extractor, tool router, and path validator are good candidates for deepening
domain-modeling	Establishing precise vocabulary for the project's domain concepts (Turn, LlmOut, ToolCall, budget, observation)
ubiquitous-language	Extracting and standardizing domain terminology across the code and documentation
testing	Setting up a test harness for the unit tests documented in approach.md Section 2
security	Reviewing or extending the AppArmor/nftables security configuration or creating security diagrams

diagram	Creating or updating Mermaid architecture/flow diagrams
implement	Implementing new features from a spec
spec	Turning vague feature requests into precise specs before implementation

Key Numbers

Constant	Value	Why
Context budget	12,288 bytes	Overhead for prefixes counted, exact fit for 2048-token model
Max read	256 KB	`take(MAX_READ_SIZE)` on file reads
Max write	1,048,576 bytes	Enforced before write and after edit
Max grep matches	50	Prevents context flooding
Max grep scan	262,144 bytes	Stops at 256KB of file content
Max dir entries	100	Bounded listing
Inner turn limit	20	Prevents infinite tool loops
Error limit	3	Consecutive parse errors abort turn
HTTP retries	3	Per-request with 1s backoff
Health check retries	5	Startup with 2s backoff
Observation max	4,096 bytes	Truncated safely at char boundaries

Critical Constraints

- Three crates only:

```
minreq
(with
  json-using-serde
),
serde
(with
  derive
),
serde_json
. Plus
  ctrlc
```

.

- **No async runtime:**

No tokio. Everything is synchronous.

- **No shell execution:**

All tools are Rust

std::fs

calls.

- **12KB context budget:**

Not 16KB. The

assembled_len()

function counts every byte including prefixes and newlines.

- **lto = "fat"**

in release profile matches Section 5.2.

- **--no-mmap + --mlock**

for llama.cpp on HDD no page-in latency.

- **AppArmor capability ipc_lock,**

not

mlock,

AppArmor 2.x compatibility.